Rochas.AI.Manager — Guia do Iniciante (rev. 2.2)

Data: 2026-10-29
Público: quem chega agora no projeto (dev, tester, operador do lab).
Documento irmão: `Rochas.AI.Manager-Arquitetura v2.md` (rev. 2.2) — o "porquê" e o "onde".
Regra editorial do projeto: capacidade só é "concluída" com **código + teste + evidência**. Este guia nunca promete o que não tem prova.

---

1. Visão geral e estado real

O AI.Manager é a plataforma .NET 10 que **concentra** o ciclo de modelos locais: catálogo, harness versionado, testes (TestBench/WAR), treino, relatórios e aprovação. O `OpenCodeBridge` (OCB) é **doador**: recebeu as correções de harness até aqui e será congelado depois da paridade.

Item	Valor medido em 2026-10-29
HEAD do AI.Manager	`537fec1` (2026-10-26 02:47)
HEAD do OCB (doador)	`3d8bd58` (2026-10-29 21:00) — **127 commits à frente** desde a última absorção
Build	`dotnet build Rochas.AI.Manager.slnx -c Release` → **0 erros / 113 avisos** (NU1902/NU1903 em `Rochas.AI.Manager.Tests` + CS9113/CS8625/CS0219)
`Bridge.Tests`	**144 PASS**
`AI.Manager.Tests`	**105 PASS + 1 SKIP**
Projetos	13 de produção (7 núcleo + 6 `Bridge`) + 2 suítes xUnit

O que isto quer dizer na prática: o produto funciona e tem prova; a **indução** (o que faz o modelo pequeno usar tools direito) está no OCB e **não** chegou inteira ao `AI.Manager.Bridge`. Fechar essa distância é a prioridade — e ela se fecha **como dado**, não como cópia de código.

2. Mapa mental do repositório

```
src/Rochas.AI.Manager.Domain/            # entidades, invariantes, TestBench (planos/cenários/runs
src/Rochas.AI.Manager.Application/       # serviços, tools handlers, ManifestBuilder
src/Rochas.AI.Manager.Infrastructure/    # AppDb, ReportService, Seeding (seeders+assets), TestBer
src/Rochas.AI.Manager.TrainingEngine/    # backend numérico (baseline MLP) + runner de processo
src/Rochas.AI.Manager.Core/              # SDK guarda-chuva
src/Rochas.AI.Manager.Api/               # REST/Swagger + proxy /v1 legado
src/Rochas.AI.Manager.Web/               # MVC, views, /health (sem seeder: chama a Infrastructure
src/Rochas.AI.Manager.Bridge/            # núcleo protocol-only (Responses↔chat, harness, anti-rep
src/Rochas.AI.Manager.Bridge.Contracts/  # manifesto, baseline, validação, hash
src/Rochas.AI.Manager.Bridge.Client/     # HTTP/SSE tipado
src/Rochas.AI.Manager.Bridge.Hosting/    # flags/env, DI, bootstrap
src/Rochas.AI.Manager.Bridge.Api/        # host REST+Swagger (/v1, /api/tools, /api/harness, /api/
src/Rochas.AI.Manager.Bridge.Runner/     # host CLI (serve/manifest)
Data/Seeds/                              # baselines versionados (war-baseline.json)
docs/ai-manager/                         # arquitetura v1/v2, guias, roadmap, pendências, PDFs
```

Regra de dependência que **não** se negocia: `Bridge` ↔ `Bridge.Contracts` **nunca** referenciam ASP.NET, CLI, `Domain` ou `Application`.

3. Conceitos básicos (o vocabulário mínimo)

- **Model Artifact / Dataset / Experiment / TrainingJob / Card**: o ciclo de treino (Draft → Approved → Deprecated; Planned → Running → Completed).
- **Harness Profile → Version → Attachment**: instruções versionadas e imutáveis depois de aprovadas.
- **MainHarness (base)**: texto enxuto que substitui o system do cliente na **raia bridge**. Preserva `<env>`, `<available_skills>`, `Today's date:` e instruções pequenas do cliente.
- **HarnessTails (cauda)**: o **delta** do canal (`Behavior` + `Tools`, e a `FsMini` só na fase F#). Base e cauda **coexistem**.
- **TailScope**: `managed` (default; todos menos o CLI do opencode), `all` (também no opencode), `off` (nenhuma).
- **Raia bridge × raia opencode**: o discriminador é o **cliente** (`User-Agent` / `X-Bridge-Mode`), nunca o sufixo do modelo.
- **Enunciado**: o **contrato observável** do teste (HTTP, transação, artefato). Mecânica de tool/harness nunca entra no enunciado.
- **Cenário (`BridgeScenario` / `test_scenarios`)**: o enunciado persistido **como dado** (prompt, workspace, tools, loop, assertions).
- **Manifesto (`BridgeManifest`)**: plano imutável e hashable que o runner executa; envelopado por `BridgeBaseline` (commit de origem + runners Q4/Q3).
- **Gate Q4 / espelho Q3**: `:4124` decide PASS; `:4125` só depois do verde.
- **Nudge / Trip**: aviso ao modelo após N falhas no mesmo alvo (soft 3) e encerramento do turno sem chamar o modelo (hard 6).
- **Salvage**: resgate de uma tool call com args inválidos — remove a assistant quebrada do histórico e devolve **um** aviso com exemplos válidos.
- **Budget de passos**: teto medido de tool calls por cenário (ex.: 24 no `E-ent-redis`); estourar o teto é falha de orçamento, não de assertiva.

- **Reset × restart**: contadores/painel zeram por `POST /api/diag/reset`; restart é só para binário trocado ou pool HTTP novo.

4. Modos de harness — as regras que evitam 80% dos bugs

1. **Raia bridge exige MainHarness.** `--mode bridge` + `--main-harness 0` é combinação **inválida** (não sobe, exit 2). Quem quer o harness nativo do opencode usa `--mode opencode`.
2. **Cauda de tools é obrigatória quando o request declara `tools`.** Sem ela o modelo perde o contrato `read/write/edit/glob` e devolve JSON como texto.
3. **Request sem `tools` não recebe cauda de tools.** Vira `DefaultBehaviorNoTools` (texto com o entregável **completo**, nunca "plano de implementação").
4. **Tails seguem o modo** (`BridgeMode.Tails` + `tail-scope`), não a raia do MainHarness.
5. **Enunciado sem tools** termina com `ENTREGUE O ARTEFATO COMPLETO EM TEXTO, SEM TOOL CALLS`. A frase `MOSTRE-ME COMO MODO PLANO EM TEXTO` é aceita por compatibilidade, mas o modelo a lê como "documento de plano" — não usar em enunciado novo.
6. **Perfil por role**: `plan`/`orch` 0.3 (thinking on), `build`/`scripter`/`exec` 0.1 (off), `moe` 0.5. Temperatura é dado do perfil, nunca texto no prompt.

5. Indução como dados — a decisão central da rev. 2.2

O OCB aprendeu, entre 2026-10-26 e 10-29, que **quase todo defeito de agente pequeno é defeito de indução**: dica que mostra literal errado, enunciado que dita mecânica, breaker cego a falha de compilador, path relativo confundido com `cd`. A correção certa é **mecanismo único no Core + texto como dado**.

Cada peça abaixo precisa existir **duas vezes**: como fonte de verdade no OCB (congelado, com prova) e como **linha de seed** no AI.Manager (dado versionado, com hash).

Peça de indução	No OCB (hoje)	No AI.Manager (alvo = dado)
Núcleo base	`Core/MainHarness.cs`	`HarnessProfile` versão `mode=ClientNative` + anexo `Nucleo`
Cauda de comportamento	`HarnessTails.DefaultBehavior` / `DefaultBehaviorNoTools`	anexo `Behavior` (duas variantes: com/sem tools)
Cauda de tools	`HarnessTails.DefaultTools`	anexo `ToolsUsage`
Mini tutorial F#	`HarnessTails.FsMini` (tabela 1 linha/código FS)	anexo `FsMini` (aplicado só quando `language=fs`)
Help curado por tool	`Core/ToolHelp.For(...)`	tabela `tool_help` (texto + schema + hash)
Aviso/nudge/trip	`BridgeNotice`, `ToolsAntiRepeat.NudgeNotice/TripNotice`	tabela `harness_notices` (`kind`, `tag`, `text`)
Aviso de salvage	`ToolCallRepair.Nudge` + `Examples` + `ToolHelp`	tabela `harness_notices` (`kind=salvage`) + exemplos validados por `JsonNode.Parse`
Reparo F# (forma certa)	`Core/FsSyntaxRepair` (mede na fonte)	catálogo `hint_catalog` (`fs_code`, `symptom`, `hint`, `probe=ok`) + algoritmo no Core

Peça de indução	No OCB (hoje)	No AI.Manager (alvo = dado)
Enunciados (belts, `E-*`, missions)	`E2EBelts*Tests`, `E2EMissionTests`, `E2EEnt*`	`test_scenarios` (prompt/system/tools/loop/assertions) via `war-baseline.json`
Perfis de runner	flags/`appsettings`	`runner_profiles` (Q4 gate / Q3 espelho)
Perfis de role/temperatura	`Core/RoleProfiles.cs`	seed de `ModelBinding` + `Agent`

Regra de ouro da absorção: nenhum texto de indução nasce no `AI.Manager.Bridge` "porque é mais rápido". Ele entra no seed, com `source_commit` do OCB, e o Core só consome. Literal no código é **fallback de emergência** e precisa de teste de igualdade contra o dado semeado.

5.1 Por que não copiar o código e seguir

Copiar `HarnessTails.cs` / `FsSyntaxRepair.cs` cria **duas fontes** que divergem na primeira edição — foi exatamente o que aconteceu entre `2.1` e hoje (127 commits). O desenho correto é:

- **algoritmo** (medir erro, decidir ramo, contar falhas) fica no Core;
- **texto** (o que o modelo lê) fica no dado versionado;
- **paridade** é provada por hash: `sha256(texto semeado) == sha256(literal do OCB em source_commit)`.

5.2 Formato do seed (resumo)

```
{
  "schemaVersion": "1.0",
  "sourceCommit": "3d8bd58",
  "sourceRepository": "rocha83/OpenCodeBridge",
  "harness": {
    "nucleo":     { "text": "...", "sha256": "..." },
    "behavior":   { "withTools": { "text": "...", "sha256": "..." },
                    "withoutTools": { "text": "...", "sha256": "..." } },
    "tools":      { "text": "...", "sha256": "..." },
    "fsMini":     { "text": "...", "sha256": "..." },
    "notices":    [ { "kind": "breaker", "tag": "BRIDGE-BREAKER", "text": "...", "sha256": "..." }
    "toolHelp":   [ { "tool": "write", "text": "...", "sha256": "..." } ],
    "hints":      [ { "fsCode": "FS0010", "symptom": "Unexpected symbol ':' in binding", "hint": "
  },
  "manifest": { "stages": [ "..." ] },
  "runners": [ { "id": "q4", "isGate": true }, { "id": "q3", "isMirror": true } ]
}
```

- A **chave natural** do seed é `(source_commit, manifest_hash)`; reexecutar não duplica.
- Plano `Approved` **não** é sobrescrito; atualização de harness entra como nova versão `Draft`.
- Hash divergente **reprova** com mensagem explícita (nunca "corrige sozinho").
- O hash do manifesto é canônico (`ManifestCanonicalizer`) e não inclui o envelope.

5.3 O que ainda é código (e deve continuar sendo)

Contagem de falhas, normalização de alvo, BFS de path, parse de JSON/args, seleção de ramo por código FS, orçamento de passos, breaker soft/hard. **Dado é o que o modelo lê; código é o que decide.** Misturar os dois é o defeito que gerou o `ToolRetryEscalation` (criado e revertido por duplicar o breaker).

5.4 Onde cada coisa mora (não errar a camada)

Responsabilidade	Camada/projeto	Proibido
Schema, repositórios, **seeders e assets de seed**	`Infrastructure` (`Seeding/`, `TestBench/`)	seeder em `Web/Data` ou em controller
Boot do host	`Web` / `Api` chamam `SeedAsync()` e seguem	controller/view criando catálogo de harness
Execução de cenário/manifesto	`Bridge.Runner` (**console**)	runner dentro da Web ou lendo SQLite
Mecânica de indução (contar, medir, decidir)	`Bridge` (Core protocol-only)	regra duplicada no consumidor
Texto de indução e enunciados	baseline em `Data/Seeds/` → banco via seeder	literal novo só no código
Contrato/validação/hash	`Bridge.Contracts`	ASP.NET no Contracts

> Dívida declarada: hoje 5 seeders ainda moram em `Web/Data` (`HarnessSeeder`, `HarnessScenarioSeeder`, `ScenarioTemplateSeeder`, `BridgeFeatureSeeder`, `WorkflowSeeder`). A migração para `Infrastructure/Seeding` é a **Fase 0.5** do roadmap.

6. Enunciados como dados

- Todo cenário vive em `test_scenarios` (prompt, `system_prompt`, `workspace_json`, `model_json`, `tools_json`, `loop_json`, `assertions_json`, `tags`, `weight`).
- Nada de regra de belt em C#: o seed `war-baseline.json` carrega prompt, tools, loop e assertions.
- Assertivas são **estruturais** quando o nome é ambíguo (ex.: detalhe = `Items` **ou** `ProductId` + `Qty`; paginação = `OFFSET` **ou** `Skip`).
- O enunciado declara **problema + contrato observável**; proibido ditar `MapGet`, `useEffect`, "não use edit", "gado PRIMEIRO" ou qualquer mecânica.
- Qualquer correção de mecânica (nudge, help, path, repair) **não** afrouxa assertiva: corrige-se o Core/dado, nunca o aceite.

7. Passos para eficiência na pipe de desenvolvimento

1. **Formalize o perfil**: `HarnessProfile` v1 a partir do núcleo + tails, sem hardcode.
2. **Lint bloqueante**: modelo, porta, contexto, temperatura, caminho e limites numéricos não entram no texto (`HarnessLinter`: 6 padrões).
3. **Preview byte-idêntico**: confira o system final exatamente como será enviado (mesmo hash).
4. **Teste no banco de provas**: um enunciado canônico por faixa; colete arquivos, `rc`, tokens/s, duração.
5. **Orçamento antes do palpite**: meça `calls=` e `steps` na janela do teste; teto é medido (24 no `E-ent-redis`), não escolhido.
6. **Fail-fast sempre**: parar no primeiro FAIL. `--no-stop-on-fail` só para medir placar completo, com consciência.
7. **Reset em vez de restart**: `POST /api/diag/reset?probe=1&files=1&panel=1` no início da bateria; restart só para binário/pool.

8. **Aprove e publique**: versão imutável, checksum, aprovação explícita.
9. **Registre runs**: perfil, modelo, hash e resultado para comparação de regressões.

8. Como rodar e verificar

```
# build de integridade
dotnet build Rochas.AI.Manager.slnx -c Release          # 0 erros

# suítes
dotnet test tests/Rochas.AI.Manager.Bridge.Tests -c Release   # 144 PASS
dotnet test tests/Rochas.AI.Manager.Tests -c Release          # 105 PASS + 1 SKIP

# hosts
dotnet run --project src/Rochas.AI.Manager.Api                # Swagger em Development
dotnet run --project src/Rochas.AI.Manager.Web                # MVC + /health
dotnet run --project src/Rochas.AI.Manager.Bridge.Api -c Release -- \
  --port 4151 --upstream http://127.0.0.1:4100 --model Qwen3.5-9B
dotnet run --project src/Rochas.AI.Manager.Bridge.Runner -c Release -- \
  manifest --manifest Data/Seeds/war-baseline.json --workspace /tmp/opencode --model smoke
```

Endpoints úteis do host: `GET /health`, `GET /api/status`, `GET /api/harness`, `GET /api/metrics`, `GET /api/tools`, `POST /api/tools/execute`, `POST /api/permission/check`, `POST /api/manifest/validate`, `POST /v1/responses`, `POST /v1/chat/completions`.

9. Troubleshooting por sintoma (o que já foi medido no OCB)

Sintoma	Causa real	Mecanismo
Modelo repete o mesmo `edit` com `oldString` obsoleto	breaker não via falha de tool (`/api/tools/execute` não passa pelo chat)	consumidor chama `ToolsAntiRepeat` / `ToolsBreaker` após cada resultado
Loop de compilador sem nudge/trip	`FailureMarkers` não casava `exit 1: ... error FS0751`	marcadores cobrem `exit [1-9]`, `error fs/cs/msb/fsi\d+`, tracebacks
Loop intercalado (edit OK, build falha) não contava	streak consecutivo zerava no sucesso	falha por **alvo mais frequente na janela**, sucesso intercalado não zera
Modelo copia a forma errada da dica	dica exibia literal inválido	"a dica ensina a forma CERTA e nunca exibe a errada" + probe de compilação
`F# primos = 0` / `[i | i <- ...]`	sintaxe de outra linguagem / divisores começando em 1	`FsMini` (1 linha por código FS) + `FsSyntaxRepair` medido na fonte
`edit` / `read` "arquivo não encontrado" com path errado	`cd` do shell não muda o path de tool	`WorkspaceGuard.Locate` devolve o caminho real (BFS depth 4)
`JSON invalido/truncado` em loop	args quebrados por anti-repeat ou modelo	janela própria de args (1024) + salvage com 3 exemplos válidos e help curado
Modelo entrega "Plano de Implementação" em vez de código	leu "modo plano" como documento	`DefaultBehaviorNoTools` + frase de fechamento correta
Bridge devolve JSON de tool como **texto** e trunca	request sem `tools`, mas com cauda de tools	`Compose(..., hasTools:false)` troca a cauda
Painel parado durante a bateria	driver publicava só `step_end`	`step_start` publicado como `RUN`

Sintoma	Causa real	Mecanismo
Contadores acumulados / gráfico picado	reset por step zerava a curva	reset **uma vez por bateria** (`?probe=1&files=1&panel=1`)
`${...}` POST sem corpo → 411	`HttpListener` exige `Content-Length`	clientes mandam corpo vazio explícito
PASS isolado, FAIL na bateria (ou vice-versa)	variância cognitiva do 4B + binário sem rebuild	guard de rebuild + 3 verdes consecutivos por nível

10. Boas práticas (resumo operacional)

- Nunca hardcode de infraestrutura no harness.
- Toda verificação que decide PASS/FAIL vive em **C#**, com teste e evidência; bash é conveniência.
- Sempre termine com build de integridade verde.
- Evidência real (log/teste) em cada run; "deve funcionar" não é entrega.
- Segmentação em micro-enunciados para modelos menores.
- DRY antes de criar: procure o artefato existente (`grep` / `git log -S`) e complete-o.
- Versionamento imutável e aprovação explícita.
- Um lugar só para cada texto de indução (dado), um lugar só para cada mecânica (Core).

11. Referências

- `Rochas.AI.Manager-Arquitetura v2.md` (rev. 2.2) — §9 (harness), §9.7 (indução como dados), §10.5 (paridade OCB → Bridge).
- `BRIDGE-TRANSPORT.md` — protocolo, anti-repeat, breaker, shim, hosts.
- `F1-F2-STATUS.md` / `ROADMAP-FIM-MIGRACAO-Q4.md` / `PLANO-APROXIMACAO-URGENTE.md` — execução do TestBench.
- `REPORT-PENDINGS.md` — catálogo R1–R30.
- `../OpenCodeBridge/HISTORY.md` e `docs/DRIVER-FIXES.md` — sintoma → causa → prova de cada mecanismo de indução.
- Swagger da API: `/swagger`.